

Highlights

Evaluating Stream Mapping Strategies for MQTT over QUIC

Fabricio Bracht

- Three stream mapping strategies for MQTT over QUIC evaluated
- The transport, not stream mapping, sets the HOL blocking level at 5% loss
- Control-only mapping has the lowest QUIC latency tail at 5% loss and 500 msg/s
- QUIC delivers 1.3–1.8× TCP+TLS 1.3 throughput under 1–10% per-packet loss
- Per-buffer loss emulation inflated QUIC throughput gains to 2–3×

Evaluating Stream Mapping Strategies for MQTT over QUIC

Fabricio Bracht^{a,*}

^aIndependent Researcher,

ARTICLE INFO

Keywords:

MQTT

QUIC

IoT

head-of-line blocking

stream multiplexing

ABSTRACT

MQTT, the dominant IoT messaging protocol, is conventionally carried over TCP, where head-of-line (HOL) blocking couples logically independent topic flows: a single packet loss delays delivery across all topics sharing a connection. Three stream mapping strategies, namely control-only, per-topic, and per-publish, each offer a different point in the isolation-overhead tradeoff space for MQTT over QUIC. We implement them in `mqtt5`, an open-source MQTT library whose broker-autonomous per-topic delivery-stream mapping is absent from existing brokers, and evaluate them across four experiments on GCP infrastructure under controlled network impairment. Using a density-corrected spike-isolation metric and per-topic tail latency to quantify HOL blocking, we find that the transport, not stream mapping, sets the level of HOL blocking at heavy (5%) loss: all three strategies keep the median per-topic tail below TCP at 5% loss (control-only 132 ms versus TCP's coupled 351 ms). Multistream adds spike-timing isolation, but as a bounded tradeoff rather than a free benefit: per-topic's isolation holds only at small topic counts, reaching single-stream coupling from about four topics, while per-publish stays isolated, at a tail latency that depends on loss and rate. On a single connection the strategies differ ninefold in publish rate, but we show that gap is set by two QUIC transport constants, the concurrent stream limit and the per-stream receive window, rather than by the mappings themselves. At our 8-topic, 500 msg/s reference point, control-only, adding the least packet overhead, has the lowest median and 90th-percentile tail among the QUIC strategies at 5% loss; at 2% it is indistinguishable from per-topic and at 1% it is the highest of the three, and at 125 msg/s under 5% loss per-publish is lowest, so which strategy has the lowest tail depends on both loss and rate. Under loss, QUIC delivers 1.26× the throughput of an encryption-matched TCP+TLS 1.3 baseline at 1% loss, rising to 1.78× at 10%, once the emulated loss removes single packets; applying the same loss per kernel buffer on the sending host inflates that advantage to 2–3×. These results provide concrete strategy selection guidelines for deploying MQTT over QUIC in IoT environments.

1. Introduction

MQTT is the dominant messaging protocol for Internet of Things (IoT) deployments, widely adopted for device-to-cloud communication across industrial, smart city, and consumer IoT platforms [17]. MQTT's publish-subscribe model, lightweight framing, and multiple quality-of-service levels make it well-suited for constrained devices and unreliable networks [18].

MQTT requires a reliable, ordered transport, and is most commonly carried over TCP. This standard transport, however, introduces a fundamental limitation: *head-of-line (HOL) blocking* [21]. When an IoT device publishes to multiple topics over a single TCP connection, as is common for devices reporting temperature, humidity, GPS, and status simultaneously, a packet loss on any one topic's data blocks delivery of *all* topics until retransmission completes. This creates artificial coupling between logically independent data flows, increasing tail latency and reducing the responsiveness of multi-topic workloads.

QUIC [9], the transport protocol underlying HTTP/3, addresses HOL blocking through *stream multiplexing*: multiple independent streams share a single connection, each with its own flow control and in-order delivery, while loss detection and congestion control remain per-connection [8]. A lost packet blocks in-order delivery only on the streams whose data it carried, although the connection-wide congestion

response can slow all of them. This property has been extensively studied for web traffic [13, 10, 14, 24], but its application to IoT messaging protocols remains largely unexplored.

Running MQTT over QUIC is not simply a matter of replacing the TCP socket. The question of *how* to map MQTT's control and data packets onto QUIC streams admits multiple answers, each with different performance characteristics. A control-only approach keeps all data on one stream, as TCP does, gaining QUIC's security and connection management benefits but not HOL blocking elimination. A per-topic approach maps each MQTT topic to a dedicated QUIC stream, intended to provide topic-level isolation. A per-publish approach creates a new stream for each PUBLISH, maximizing isolation at the cost of stream management overhead. We use these three names throughout.

In this paper, we implement and evaluate three stream mapping strategies for MQTT over QUIC in `mqtt5`¹, an open-source MQTT library, across four experiments on GCP infrastructure under controlled network impairment.

Our contributions are:

1. The first implementation of *broker-autonomous* per-topic and per-publish delivery-stream mapping for MQTT over QUIC. The operating modes are defined by EMQ's *MQTT-next* specification [23]; we implement the broker side that specification describes but that no production broker realizes, in that the broker opens its

*Corresponding author

 fabracht@gmail.com (F. Bracht)

ORCID(s):

¹<https://github.com/LabOverWire/mqtt-lib>

own QUIC streams and maps each subscriber's deliveries per topic rather than only mirroring deliveries back onto client-initiated streams.

2. The first vendor-independent, controlled, statistically-treated evaluation of MQTT-over-QUIC operating modes, namely control-only, per-topic, and per-publish, under emulated network impairment, quantifying head-of-line blocking and throughput.
3. An argument for why QUIC stream multiplexing, rather than N parallel TCP connections, is the concurrency mechanism available to an MQTT client: the v5.0 session-takeover rule evicts an existing connection when a second presents the same client identifier [18], so one identity cannot hold the parallel connections that would otherwise supply per-topic concurrency.
4. An open-source implementation and a reproducible benchmarking harness.

The remainder of this paper is organized as follows. Section 2 covers MQTT, QUIC, and related work. Section 3 describes the stream mapping architecture. Section 4 details the implementation. Section 5 presents experimental results. Section 6 discusses strategy selection guidelines and future directions. Section 7 concludes.

2. Background and Related Work

2.1. MQTT Protocol

MQTT [18] provides three quality-of-service levels (QoS 0: at-most-once, QoS 1: at-least-once, QoS 2: exactly-once), retained messages, last will and testament, and topic-based filtering with wildcard support. MQTT v5.0 added user properties, shared subscriptions, and enhanced authentication. The protocol's minimal framing overhead (2-byte minimum header) makes it well-suited for constrained devices.

2.2. QUIC Transport Protocol

Beyond the stream multiplexing described in Section 1, QUIC [9] provides several features relevant to IoT messaging:

Integrated security. QUIC integrates TLS 1.3 into the transport handshake, achieving a 1-RTT connection establishment that includes both transport and cryptographic setup [22]. This compares favorably to TCP's 1-RTT handshake plus TLS's additional 1–2 RTTs.

Loss detection. QUIC uses per-packet sequence numbers (unlike TCP's per-byte) and acknowledgment-based loss detection with probe timeout (PTO) for tail loss [8]. This enables more precise loss recovery than TCP's retransmission timeout mechanism.

2.3. Related Work

Prior work on MQTT over QUIC falls into two strands, which we take in turn.

Implementations and evaluations. Kumar and Dezfouli [12] implemented MQTT over QUIC using a single bidirectional stream, demonstrating reduced connection overhead (up to 56% fewer packets exchanged with the broker) and lower delivery latency (up to 55%) compared to MQTT

Table 1

MQTT-over-QUIC stream mapping across implementations. The operating modes originate in EMQ's *MQTT-next* specification [23]; implementations differ chiefly in *where* topic-to-stream mapping occurs. The `mqtt5` column is verified against its source; the EMQX and NanoMQ columns against their published documentation and source. For EMQX and NanoMQ, multistream is a client-initiated capability rather than default behavior: EMQX runs single-stream unless the application opens data streams through its client API, and NanoMQ's multistream exists only in its outbound bridge and is disabled by default. The "Published evaluation" row records where each implementation's performance numbers come from: EMQX's are published by its own vendor, Kumar's are peer-reviewed, and `mqtt5`'s are the present manuscript, whose implementation and evaluation share an author; the open implementation and reproducible harness are intended to offset that. The EMQX and NanoMQ columns describe the sources and documentation cited here at the time of writing and may change across releases.

	EMQX [5]	NanoMQ [6]	Kumar [12]	mqtt5
MQTT over QUIC	✓	✓	✓	✓
Multistream (client-initiated)	✓	✓	✓	✓
Client topic→stream mapping	—	✓ (bridge)	—	✓
Broker-initiated delivery streams	—	—	—	✓
Broker topic→stream mapping	—	—	—	✓
Published evaluation	vendor	—	peer-reviewed	this work

over TCP across wired and wireless testbeds. Fernández et al. [7] evaluated MQTT over QUIC under emulated wireless conditions (WiFi, LTE, satellite), finding that QUIC maintains stable throughput under packet loss while TCP degrades. Neither study explored stream multiplexing for topic-level HOL blocking isolation.

The multistream operating modes, and where mapping happens. Mapping MQTT traffic onto multiple QUIC streams is established industrial practice, and its taxonomy predates this work. EMQ's *MQTT-next* specification [23] defines three operating modes, namely Single Stream, Simple multistreams, and Advanced multistreams, and names "mitigate HOLB" as the multistream benefit; it also defines the flow header and, for the advanced mode, broker-initiated streams. The single/per-flow/per-frame stream-mapping taxonomy is older still: the 2017 RTP-over-QUIC draft [20] mapped media onto QUIC streams at exactly these three granularities. We therefore do not claim to originate these strategies or the flow header. Existing implementations, however, realize only the *client-driven* portion of the design. EMQX [5] accepts multiple client-initiated streams but opens none of its own, so any delivery-side isolation is driven entirely by the client rather than the broker; in a default session its own client uses only the control stream. NanoMQ [6] pairs configured topics to streams via a per-topic stream identifier in its bridge configuration, but only within its outbound bridge *client*, and only as a default-off, work-in-progress feature. Table 1 summarizes where each implementation performs mapping.

The HOL blocking problem in multiplexed protocols has been studied extensively in the context of HTTP/2 [1] and HTTP/3 [2]. Langley et al. [13] demonstrated QUIC's

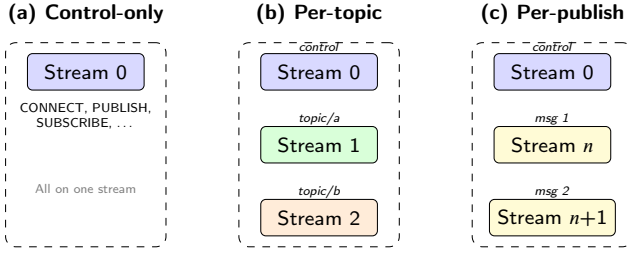


Figure 1: The three stream mapping strategies. (a) Control-only multiplexes everything on stream 0. (b) Per-topic assigns a dedicated stream per topic, intended to provide topic-level isolation. (c) Per-publish creates a new stream for each PUBLISH, providing message-level isolation.

benefits at Internet scale, reporting reduced search latency and video rebuffering across Google’s infrastructure. Subsequent empirical work found nuanced results: Kakhki et al. [10] showed that QUIC’s practical effect on web page loading is modest despite reducing transport-layer HOL blocking, and Marx et al. [14] demonstrated that implementation-level decisions (prioritization, frame scheduling) further affect the degree of isolation achieved. Yu and Benson [24] corroborated these nuanced findings through production-scale measurements. Our work applies stream multiplexing to the messaging domain, where the “resource” granularity is the MQTT topic rather than the HTTP request, and independent topic flows may benefit more directly from stream isolation.

To our knowledge, no MQTT-over-QUIC *broker* autonomously maps topics to server-initiated delivery streams, which is the mechanism we implement and evaluate, and no peer-reviewed study independently evaluates and compares these operating modes under controlled network impairment. This paper addresses that gap.

3. Architecture

The `mqtt5` library maps MQTT protocol operations onto QUIC streams using configurable *stream mapping strategies*. The architecture decouples the MQTT session layer from the transport layer, allowing the same MQTT implementation to operate over TCP, TLS 1.3, WebSocket, or QUIC with different stream configurations.

3.1. Stream Mapping Strategies

We define three strategies for mapping MQTT control and data packets onto QUIC streams, each offering a different point in the isolation–overhead tradeoff space.

Control-only (Fig. 1a) uses a single bidirectional QUIC stream for all MQTT traffic. This is structurally equivalent to running MQTT over a single TCP connection and provides no stream-level HOL blocking mitigation, though it still keeps a far lower tail than TCP at 5% loss (Section 5.4). It serves as the QUIC baseline, isolating the effect of the QUIC transport as a whole from stream multiplexing.

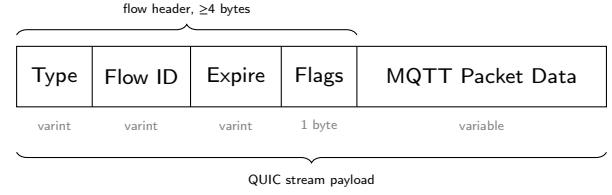


Figure 2: Data flow header format, as defined by the MQTT-next specification [23]: variable-length integer Type, Flow ID and Expire fields and a one-byte Flags field, four bytes at minimum. On the client’s publish path the header is optional and we leave it disabled throughout; on the broker’s delivery path it is always written, so every server-opened stream under per-topic and per-publish carries one.

Per-topic (Fig. 1b) assigns each MQTT topic to a dedicated QUIC stream. Control packets (CONNECT, SUBSCRIBE, PINGREQ, etc.) use stream 0, while PUBLISH packets are routed to topic-specific streams. A hash map maintains the topic-to-stream mapping, creating streams lazily on first publish. This provides topic-level HOL blocking isolation: a retransmission delay on one topic’s stream does not block delivery on other topics’ streams, though the isolation narrows as the topic count grows, for the reason Section 5 measures.

Per-publish (Fig. 1c) creates a new unidirectional QUIC stream for each PUBLISH packet. This provides the finest-grained isolation, because every message is independent, but at the highest overhead cost, as each message requires opening a new QUIC stream.

3.2. Flow Header Protocol

Under per-topic and per-publish, the receiver must know how to demultiplex the data arriving on each non-control stream. For this purpose the MQTT-next specification [23] defines a *flow header*, placed at the start of a data stream ahead of the MQTT packets it carries, and our implementation supports it.

The header (Fig. 2) is optional. It carries a Type field distinguishing client from server data, a Flow ID that names the flow for recovery on a later stream, and an Expire interval, each a variable-length integer, followed by a one-byte Flags field whose bits are the specification’s session-persistence controls (persistent subscriptions, topic aliases and QoS state, error tolerance, and clean-start semantics) rather than per-message MQTT flags. The receiver detects the header from the first byte and parses plain MQTT framing when it is absent. The two directions differ in our implementation. On the client’s publish path the header is optional and we leave it disabled throughout, so client-side data streams carry plain MQTT. On the broker’s delivery path it is unconditional: every stream the broker opens for delivery is prefixed with one. Control-only opens none, per-topic one per topic, and per-publish one per message, so the header is itself part of the per-message overhead that Section 5 measures on that path.

3.3. Broker Integration

The client and broker configure their stream strategies independently. On the client side, the application selects a strategy (control-only, per-topic, or per-publish) before connecting; this governs how outbound PUBLISH packets are dispatched onto QUIC streams. On the broker side, a server-wide delivery strategy governs how the broker sends messages to subscribers. In a typical deployment, both sides are configured to the same strategy, but the architecture does not enforce this, so a per-topic client can connect to a control-only broker, or vice versa.

For outbound delivery, the broker's stream manager maintains a per-connection topic-to-stream mapping. When configured for per-topic delivery, streams are created lazily on first use and reused for subsequent messages on the same topic, with LRU eviction to bound stream count. For per-publish delivery, the broker opens a new unidirectional stream for each message. On the receive side, the broker's QUIC acceptor classifies streams by kind and arrival order rather than by numeric identifier: the first bidirectional stream accepted becomes the control stream and carries the full MQTT packet parser, while later data streams are probed for a flow header and otherwise parsed as plain MQTT, their PUBLISH packets routed into the topic distribution engine.

4. Implementation

`mqtt5` is an open-source MQTT library written in Rust. The library provides both a client and broker, supporting TCP, TLS 1.3, WebSocket, and QUIC transports. The QUIC implementation uses Quinn [19], a Rust QUIC library built on the tokio async runtime.

4.1. Transport Abstraction

The library defines a Transport trait abstracting read/write operations over any underlying connection. For TCP and TLS, this wraps a single byte stream. For QUIC, the transport layer manages multiple streams through a `QuicStreamManager` that implements the configured strategy.

The stream manager maintains a per-topic map from topic name to its send stream, creating streams lazily on first use and reusing them for subsequent messages on the same topic. For per-publish, it opens a new unidirectional stream for each PUBLISH and closes it after sending. The initial bidirectional stream carries control packets, and under control-only it carries the PUBLISH traffic as well.

4.2. Connection Lifecycle

QUIC connections follow the same MQTT session lifecycle as TCP connections. The CONNECT packet is always sent on stream 0. After CONNACK, subsequent PUBLISH packets are dispatched according to the client's configured stream strategy.

On the broker side, the QUIC acceptor spawns a dedicated task per connection, classifying streams as described in Section 3.

4.3. Benchmarking Infrastructure

The library includes an integrated benchmarking tool (`mqtt5 bench`) that measures latency and throughput. Latency mode embeds microsecond timestamps in payloads and computes p50/p95/p99 percentiles. Throughput mode counts messages per second over a configurable duration with warmup. Both modes support configurable publisher/subscriber counts, QoS levels, payload sizes, and transport parameters.

The evaluation uses two builds of `mqtt5`. The *pricing build* is release 0.41 of the library, whose benchmarking tool can pace publishers at a fixed aggregate rate instead of letting them send as fast as they can. The *evaluation build* lacks this option; its binary checksum is recorded with the data [3]. The two builds' absolute rates differ, by amounts Section 5.5 reports, so every conclusion rests on comparisons within one build; where the two builds' results appear together, the text reports the difference, and a side-by-side comparison at 10% loss measures it directly. The pricing build produces three sets of Experiment 3 runs: the offered-load check (paced publishers and the unpaced runs they are compared against), the broker delivery-strategy runs at fixed publish rates with their unpaced counterparts, and a side-by-side comparison of both builds at 10% loss. The evaluation build produces every other result.

Network impairment is applied externally via `tc-netem`, and broker/client resource usage (RSS, CPU, network I/O) is sampled at 1 Hz during each run using `procs` monitors. To observe the loss-bearing path directly, the 5% loss condition of Experiment 2's three QUIC arms also has an instrumented arm of five runs each, with the broker sampling its own QUIC connection statistics at 10 Hz on each connection it holds, giving congestion window, losses and congestion events; the subscriber-facing connection is identified per run as the one carrying more packets. The instrumented arm uses the Experiment 2 configuration and matches the uninstrumented arm's tail ordering and level (median per-topic p99 133.7, 161.5 and 209.1 ms for control-only, per-topic and per-publish, against 131.9, 164.3 and 194.7 ms); tail figures come from the uninstrumented arm and connection statistics from the instrumented one. For Experiment 3's primary arms the impairment is instead applied on a forwarding VM placed between the broker and its clients by a policy-based route, with receive coalescing disabled so that each loss removes exactly one packet (Section 5.2).

5. Evaluation

We conduct four experiments, measuring `mqtt5`'s connection establishment, head-of-line blocking, throughput, and stream strategy behavior. All four use QoS 0, so that the comparison measures the transports' own loss recovery and nothing layered above it. QoS 1 and 2 add MQTT-level acknowledgements that travel the same impaired path, and the Receive Maximum in-flight window [18] caps unacknowledged messages, so every transport-level delay to an acknowledgement would also throttle the publisher at the MQTT layer and mix application-level flow control into

the transport comparison. On a live connection MQTT v5.0 leaves retransmission to the transport, resending unacknowledged messages only after a reconnect.

Experiments 2 and 4 set the broker's delivery strategy to match the client's. Experiment 3's main comparison holds the broker at per-topic delivery across the QUIC arms, isolating the client's publish-side mapping, and a second set of arms varies only the broker: control-only against per-topic delivery with a control-only client, and per-publish against per-topic delivery with a per-publish client. Because client and broker strategies co-vary in Experiment 2, the broker-side packet and loss counts reported there compare matched client-and-broker strategies and cannot isolate the broker's mapping.

5.1. Experimental Setup

Table 2 summarizes the common parameters. Two details behind it matter for reproduction. Experiment 3's delivered throughput, and the single-connection delivered rates quoted in Experiment 4, are the mean of the per-second delivered rate over the central 80% of the window, the first and last 10% of samples being dropped as ramp-up and drain; Experiment 4's publish rates and Experiment 2's rates are instead a message count divided by the full window. The per-packet Experiment 3 runs are randomized across loss levels, loss-application arms and transports. Network impairment is applied with *tc-netem* on the broker's egress interface in Experiments 1, 2 and 4 (Experiment 1 applies delay only), and on a forwarding VM in Experiment 3. Experiment 3's robustness arms and the per-packet variant of Experiment 2 keep the delay on the broker and apply the loss on the forwarding VM (Section 5.2). Each experiment varies one or two factors while holding others constant. Experiment 2 uses co-located publisher and subscriber on a single client VM to eliminate network variability between pub/sub; all other experiments use separate VMs.

Statistical reporting. Unless a caption states otherwise, each plotted data point aggregates 15 independent runs. The line plots show the mean with its 95% confidence interval, computed as the Student *t* interval $\bar{x} \pm t_{0.975, n-1} s/\sqrt{n}$ and rendered as capped error bars or a shaded band. The connection-latency bars of Fig. 3 are different in kind: the bar is the p50 and the whisker reaches the p95 of the within-run latency distribution, not a confidence interval of the mean. Because the delay is fixed and the Bernoulli loss rate is held constant, run-to-run variation in the throughput and strategy cells is low, so these intervals are frequently narrower than the plotted marker and can be hard to see. They are drawn at every point and never omitted; their half-width stays below 4% of the mean at the large majority of points on the throughput and strategy plots, and is visibly wider only on the density-corrected excess of Fig. 4, where the underlying values are small and most cells aggregate fewer runs (9–10; 15 for the 8-topic, 500 msg/s cells) than the 15-run default. An interval that is not discernible is present but smaller than the plotting symbol, which reflects measurement stability rather than missing data.

Table 2

Common parameters for the four experiments.

Parameter	Value
Hardware	GCP n2-standard-4 (4 vCPU, 16 GB RAM)
Topology	Co-located pub/sub (Exp 2), separate VMs (others); forwarding VM (Exp 3)
Region	us-west1-b (same zone, <1ms baseline RTT)
OS	Ubuntu 24.04 LTS (kernel 6.17.0-1008-gcp pinned for Exp 3's per-packet and paced runs)
One-way delay	0–200 ms swept (Exp 1), 25 ms (Exp 2, 4; also swept 10–50 in Exp 4), 10 ms (Exp 3)
Network emulation	<i>tc-netem</i> , Bernoulli loss: per buffer on broker egress (Exp 2, 4; Exp 1 delay only); per packet on a forwarding VM (Exp 3; Section 5.2)
QUIC library	Quinn 0.11 (Rust)
TLS	rustls 0.23 (TLS 1.3 only)
Congestion control	CUBIC (both QUIC and TCP)
Implementation	<i>mqtt5</i> evaluation build; pacing build for Exp 3 paced checks (Section 4.3)
Frame packing	Greedy (the default) in every arm
Concurrent stream limit	100 (the QUIC library default) except where Exp 4 sweeps it
Per-stream receive window	256 KB except where Exp 4 sweeps it; connection window 1 MB
Flow headers	Off on the client publish path; always present on broker-opened delivery streams
Payload size	256 bytes (Exp 2–4; Exp 1 establishes connections only)
Publish rate	500 msg/s base, swept 125–2000 (Exp 2); max throughput (others), paced 1.25K–20K in Exp 3 paced checks
Measurement	60 s (Exp 2), 70 s subscriber (Exp 3), 60 s publisher (Exp 4), 30 s (Exp 1); 5 s warmup (0 s for Exp 1)
Runs per config	15 (9–15 Exp 2 sweep; 5 Exp 2 instrumented arm; 10 Exp 4; 5–15 Exp 3 robustness and paced runs; exclusions noted where reported)
HOL spike window	10 ms (Exp 2 spike isolation)
Broker subscriber queue	10,000 messages per subscriber; QoS 0 messages are dropped when full

5.2. Emulating Packet Loss

tc-netem makes one loss decision per buffer it is handed and drops the whole buffer. With segmentation offload the kernel hands the *qdisc* buffers that carry several packets, and QUIC's datagram batching does the same for UDP, so loss applied on a sending host clusters into fewer, larger events whose size depends on the transport and the load. The *qdisc* statistics hide this, since they count the packets inside each buffer while counting a dropped buffer once;

the clustering shows up only as a drop fraction below the configured rate. Applying loss this way on the broker's egress, as Experiments 2 and 4 do, dropped 7.7% of QUIC's packets and 9.0% of TCP+TLS's at 10% configured loss in Experiment 3's fan-out configuration (three runs per transport).

For Experiment 3 we therefore apply both the delay and the loss on a separate forwarding VM, through which a policy-based route sends the broker's traffic to its clients. Receive coalescing is disabled on that VM, so each buffer its netem sees is one wire packet; a kernel probe on its enqueue path found no multi-packet buffer and a largest frame of 1474 bytes. The measured drop fraction matched the configured rate within its 99.9% binomial interval in all but 2 of 972 lossy runs. With a 99.9% interval about one run in a thousand falls outside by chance, so two is consistent with correct emulation; we excluded both (4.93% and 4.94% at 5%). The forwarding hop adds 28–58 μ s one way, which we subtract from the emulated delay, and at 0% loss routing through it changes no transport's throughput by more than 2% relative to the direct path. Loss remains Bernoulli, each packet being dropped independently with the configured probability, and applies only to broker-to-client traffic. Experiments 2 and 4 keep the broker-egress method; Section 5.4 also reports Experiment 2's reference cell under per-packet loss.

5.3. Experiment 1: Connection Establishment Latency

We measure cold-start connection latency across TCP, TLS 1.3, and QUIC at five one-way delays (0, 25, 50, 100, 200 ms) over 30 s of back-to-back cold connections per run. The 0 ms condition is the only one in which the client opens connections faster than the network paces them, so its results are shaped by client-side socket resources (ephemeral ports, `TIME_WAIT` state, accept backlog), none of which we tuned; we read that condition as a property of the test host as much as of the transport.

Fig. 3 shows that QUIC 1-RTT connection establishment matches TCP's cold-connect latency at every emulated delay of 25 ms or more. At 0 ms the comparison is dominated by local processing rather than the network: QUIC (1.55 ms) and TLS (1.59 ms) are indistinguishable and both sit about 2.4–2.5 $\times$ above raw TCP's mean of 0.64 ms, which runs no cryptographic handshake; raw TCP's runs split between 0.26–0.30 ms and 1.0–1.2 ms, so that mean describes no single run. That cell is also the only one in which connections failed, and the transports fare oppositely there: TCP lost 26.3% of its attempts and TLS 1.3 4.4%, while QUIC completed every one. At 25 ms and above no arm failed a connection. TLS 1.3 over TCP consistently adds one additional round trip. At 200 ms delay, QUIC connects in 402 ms (p50) versus TCP's 400 ms and TLS's 602 ms. The p95 whiskers are tight beyond the 0 ms case, with p95 within 2% of p50 at every delay of 25 ms or more (within 0.6% for TCP and TLS 1.3); only at 0 ms, where the handshake is bound by local processing rather than the network, is the distribution appreciably wider. The integrated handshake eliminates the sequential TCP+TLS

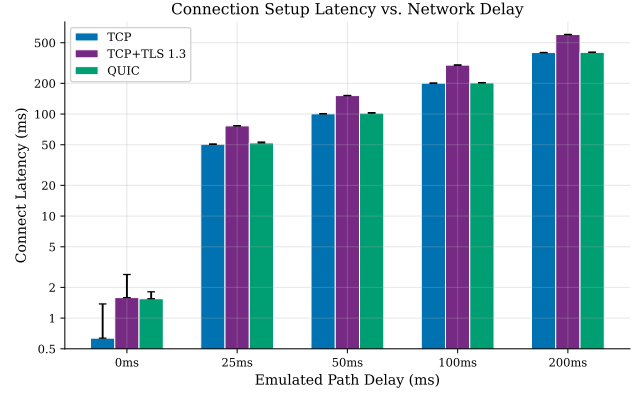


Figure 3: Connection latency (p50 bars, p95 whiskers) across network delays. QUIC's integrated TLS handshake achieves comparable latency to raw TCP despite cryptographic overhead, and outperforms the separate TCP+TLS 1.3 handshake at every emulated delay of 25 ms or more. At 0 ms, where the handshake is bound by local processing rather than the network, all three complete in under two milliseconds, QUIC and TLS are indistinguishable, and raw TCP is fastest.

penalty without sacrificing security. We did not test QUIC 0-RTT resumption, which could further reduce connection latency for returning clients.

5.4. Experiment 2: Head-of-Line Blocking

This experiment is the core motivation for MQTT-over-QUIC multistream transport. Head-of-line blocking couples logically independent topics: when packets share one ordered connection, a single loss delays every topic queued behind it [21]. We publish across many topics at once and measure whether this cross-topic propagation occurs. In each QUIC arm the broker's delivery strategy matches the client's, so the arms exercise the broker-autonomous delivery mapping as well as the client's publish mapping. Around a reference point of 8 topics and an aggregate 500 msg/s, we vary two factors independently: holding the rate at 500 msg/s we sweep the topic count from 2 to 32 topics, and holding the count at 8 topics we sweep the offered rate from 125 to 2000 msg/s; the two sweeps meet at this reference point, which we also use for the tail-latency comparison. At 500 msg/s the connection is unsaturated, so latency variation reflects head-of-line blocking rather than queue buildup, while the highest swept rates deliberately drive a lossy connection into saturation. We emulate Bernoulli packet loss at 0%, 1%, 2% and 5%. The tail-latency comparison spans all four, with 0% as an unimpaired baseline; the density-corrected timing analysis uses the 1% setting alone, since it needs spikes to be sparse (as explained below) and at 0% loss no retransmissions occur, so head-of-line blocking is absent by construction.

We measure HOL blocking directly, through the *timing* of latency spikes: does a spike on one topic co-occur on the others (coupling) or stay confined to it (isolation)? A latency spike is a message whose latency exceeds twice the median of the preceding 50 messages on its topic, and the *spike isolation ratio* is the fraction of spikes that co-occur with a spike on

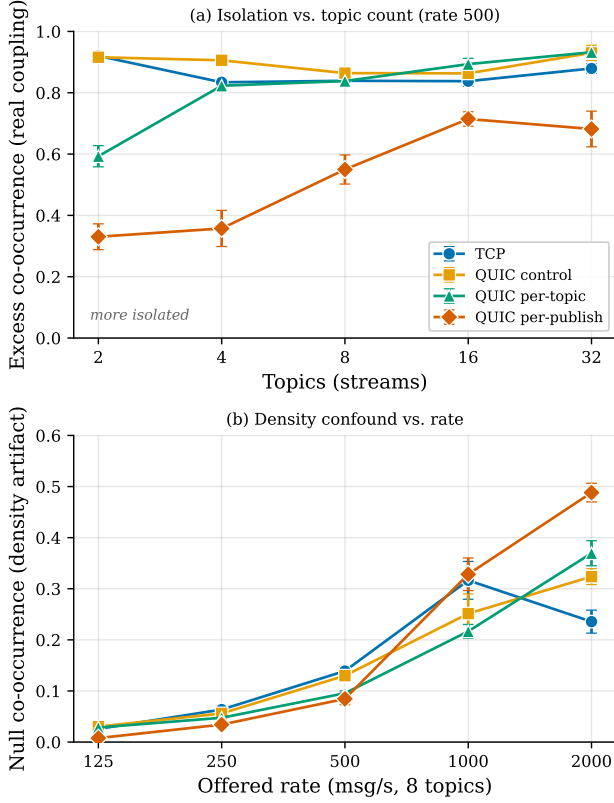


Figure 4: Density-corrected head-of-line isolation (means over 9–15 runs per cell, 15 for the 8-topic, 500 msg/s cells, 95% confidence intervals). (a) Excess spike co-occurrence (observed ratio minus a density null; lower is more isolated) versus topic count at 500 msg/s and 1% loss. TCP and control-only stay coupled at every topic count; per-publish is the most isolated; per-topic isolates only at small topic counts and reaches single-stream coupling from 4 topics on. (b) The density null grows with offered rate, so the raw ratio cannot be compared across rates or loss levels.

at least one other topic within a 10 ms window: 1.0 means every spike propagates across topics (full HOL blocking), lower values mean spikes stay confined. Because a raw co-occurrence count is inflated by spike *density* (when spikes are frequent, some coincide by chance), we report the density-corrected *excess*: the observed ratio minus a null obtained by independently circularly shifting each topic's spike train (averaged over 100 shifts), which preserves each topic's spike density and autocorrelation while removing genuine cross-topic alignment. The excess is meaningful only while the density null stays well below 1: because the observed ratio cannot exceed 1, the excess is bounded by one minus the null, and once chance coincidences become common that bound squeezes coupled and isolated strategies together. We ran the sweep at 1% and 5% loss only.

Fig. 4(a) confirms both the blocking and its isolation. The single-stream transports couple the topics: on TCP and QUIC control-only a loss's spike co-occurs across topics (excess 0.83–0.93 across the topic sweep). Splitting topics

onto separate streams decouples them, and at 2 topics per-publish and per-topic isolate (excess 0.33 and 0.59). The isolation is real but bounded: it erodes as topics are added, and from 4 topics on per-topic's excess reaches single-stream coupling (0.82 against TCP's 0.83 at 4 topics, 0.84 against 0.84 at 8), and at 16–32 topics it is at control-only's level, above TCP's, leaving only per-publish still isolating (0.55 at 8 topics). This comparison is confined to 1% loss, where the density null stays at or below 0.14 across the whole topic sweep (Fig. 4(b) shows it growing broadly with offered rate instead), so excess is well determined. At 5% loss the null is two to six times larger at 4–16 topics (up to ten times at 32), reaching 0.44–0.50 at 8 topics and 0.53–0.65 at 16, and 0.59–0.73 for the QUIC arms at 1000 msg/s, which leaves too little room below the 1.0 ceiling for excess to be separable from density. At 5% loss and 2000 msg/s the failure runs the other way, because the transports are overloaded. TCP's single connection to the subscriber carries only about 880 msg/s whether 1000 or 2000 msg/s is offered, within the loss-limited range of Mathis et al. [15] for this path (about 810–1140 msg/s for a model constant between 0.87 and 1.22, a 25 ms round trip, 1408-byte segments and about 270 bytes per message), so the rate is a property of one connection rather than of TCP in aggregate. The excess piles up in the broker's queue for that subscriber, which holds 10,000 messages and drops QoS 0 messages once full: roughly half of the offered messages are discarded there, and every delivered message waits about as long as the full queue takes to drain, roughly 12 s. A spike is defined relative to recent latency, so a retransmission stall of tens of milliseconds no longer registers against that baseline, and the detector finds no spikes at all, in every TCP run and in nine of ten per-publish runs. The observed ratio and the null are then both zero because there is nothing to count, not because the topics are isolated. TCP at 1000 msg/s is already capped at the same rate but its queue does not fill within a run; it finds no spikes in five of its ten runs and a single, unpaired spike in a sixth. Control-only and per-topic are queueing too at 2000 msg/s and lose all spikes in one and two runs respectively, so their excess is too erratic to estimate (confidence intervals of ± 0.27 – 0.28).

How that isolation shapes tail latency (Fig. 5) is governed by two effects. First, the transport, not stream mapping, sets the level: even single-stream control-only, which shares TCP's single ordered-stream structure, holds a median 132 ms at 5% loss against TCP's 351 ms. We did not isolate which transport difference is responsible. Second, the strategies carry the same message load but differ in how many packets they put on the *delivery* path, and under this loss method the tail ordering tracks that packet count. Control-only adds no per-stream framing and has both the lowest median (132 ms) and the lowest 90th-percentile tail (162 ms); per-topic follows at a 164 ms median, and per-publish, which opens a stream for every message, is worst at 195 ms (under per-packet loss it ties per-topic; see below). Both gaps are significant over the 15 runs per arm (Mann–Whitney U , two-sided: control-only versus per-topic $p = 0.006$, versus per-publish $p = 0.0003$). The broker-side statistics from the instrumented

runs (Section 4.3; medians over five runs per arm) show a chain consistent with that ordering: on the subscriber-facing connection the broker sends about 10.8K packets per run under control-only, 11.8K under per-topic and 20.9K under per-publish. The emulated loss rate is identical for all three arms, so the extra packets become proportionally more losses (540, 598 and 1076) and disproportionately more congestion events (347, 407 and 853). The congestion window responds only at the extreme: per-publish's median is measurably depressed (6.9K versus control-only's 7.7K bytes, $p = 0.008$) while per-topic's is indistinguishable from control-only's ($p = 0.69$). Under this loss method, per-publish's longer tail thus coincides with a smaller window as well as more recovery events, whereas per-topic's coincides with the extra recovery events alone. Experiment 4 shows per-publish's one-stream-per-message pattern separately cutting a single connection's publish rate roughly ninefold, through the peer's stream limit rather than through packet count, a single-connection limit that does not reach the broker's aggregate throughput when the broker delivers per-topic, although a ceiling consistent with it appears in broker-side per-publish delivery per subscriber (Experiment 3). Individual runs can spike well above their arm's median. One control-only run at 5% loss reached 342 ms, 2.6 times the arm's median of 132 ms; its second-worst run reached 168 ms, and among the fifteen runs per arm the worst multistream runs reached 174 ms (per-topic) and 254 ms (per-publish). The five instrumented and fifteen per-packet runs of this cell produced no control-only run above 159 ms (the instrumented per-publish runs reached 292 ms, 1.4 times their median). Smaller but still large outliers, 1.6–1.8 times the cell median, appear in all three QUIC strategies elsewhere in the 5%-loss sweeps at rates up to 500 msg/s, so we treat the 342 ms run as an outlier rather than as a property of control-only; fifteen runs per arm cannot estimate how often such outliers occur. Below 5% loss the ordering is different. At 0% the three sit within 0.4 ms of one another. At 1% control-only is in fact the highest of the three (58.3 ms against per-topic's 55.8 and per-publish's 54.0; $p = 0.007$ and $p = 0.003$), so finer stream granularity does help at light loss. At 2% control-only and per-topic are indistinguishable (68.1 versus 68.4 ms, $p = 0.15$) while per-publish has separated upward (82.6 ms; $p < 0.001$ against control-only, $p = 0.002$ against per-topic). At 500 msg/s, control-only's tail advantage is therefore specific to the heaviest loss we emulate; the rate sweep also shows it at 1% loss and 1000 msg/s (61 against 70 and 82 ms).

Under this loss method, coarser mapping (control-only) minimizes packet overhead and has the lowest median and 90th-percentile tail at 5% loss. Finer mapping decorrelates spike timing but adds framing packets, and per-publish's per-message streams roughly double the packet count and, under this method, raise its median. At 2000 msg/s under 5% loss it also delivers less than the other QUIC arms, 1.64K against 1.94K msg/s. That shortfall is mostly the concurrent stream limit rather than the framing: the same cell in Experiment 4's setup gives 1.61K at the 100-stream default and 1.90K once

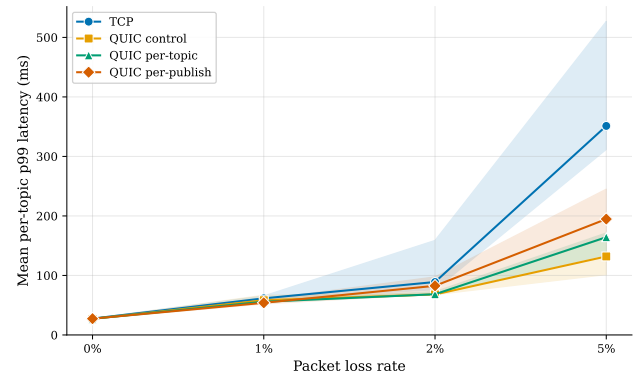


Figure 5: Mean per-topic p99 latency versus packet loss at 8 topics (line is the median over runs; shaded band spans the 10th–90th percentile of runs). At 5% loss all QUIC strategies stay below TCP, and at 2% control-only and per-topic do. At 1% per-topic and per-publish are below TCP and control-only is indistinguishable from it; in the 1% rate and topic sweeps TCP's tail is far higher at 1000–2000 msg/s and at 32 topics, and nominally (never significantly) lower in a few cells. At 5% loss control-only has the lowest median and 90th-percentile tail and per-publish the highest (under per-packet loss per-publish ties per-topic; Section 5.4); at 2% control-only and per-topic are indistinguishable and only per-publish has separated upward; at 1% control-only is the highest of the three. A single control-only run at 5% loss reached 342 ms (outside the band); Section 5.4 discusses such outliers.

the limit is raised to 1000 (three runs each). Both the isolation and the tail therefore turn on one variable, stream granularity. Because the overhead term scales with rate, the ordering crosses over: at 125 msg/s under 5% loss per-publish's extra packets are few in absolute number and it has the lowest tail of the three (96 ms, $p = 0.0002$ against control-only), whereas by 1000 msg/s it is nominally the worst (361 against 302 and 313 ms, not significant; the rate sweep uses broker-egress loss only). The spike isolation itself erodes with topic count because, at a fixed rate, a single packet increasingly coalesces several topics' data, so one loss recouples them. QUIC loss detection is per-connection rather than per-stream [8], so we could not separate per-stream recovery timing from packet count and coalescing.

Experiment 2 applies its loss with the broker-egress method of Section 5.2, so we also ran its 8-topic, 500 msg/s reference cell at 0% and 5% loss with per-packet loss on the forwarding VM (15 runs per arm; at 0% two TCP runs were excluded for retransmissions on a clean path, leaving 13). At 0% every arm stays within 0.3 ms of its level in the broker-egress runs (27.3–27.7 ms). At 5% the ends of the ordering hold: control-only remains the lowest (108 ms; $p = 0.0004$ against either multistream arm), and TCP stays far above every QUIC arm (458 ms, with runs spread from 200 to 1297 ms). Only one arm moved significantly at 5%: per-publish fell from 195 to 143 ms ($p < 0.001$) and no longer trails per-topic (141 ms, $p = 0.65$), although it still sends about 1.5 times per-topic's packets on the loss-bearing link. Its penalty against control-only shrinks from 63 to 35 ms

but remains. TCP (351 against 458 ms, $p = 0.28$), control-only (132 against 108, $p = 0.06$) and per-topic (164 against 141, $p = 0.09$) stay within run-to-run variation between the two loss methods. Control-only also has the lowest worst run and 90th percentile (146 and 138 ms, against 189 and 169 ms for per-topic and 168 and 155 ms for per-publish). This variant changes more than the loss granularity (the loss moved to the forwarding VM and the broker's delay queue ran at 100,000 packets rather than 1,000, which also changes TCP's packetization; Section 5.5), so we cannot attribute the per-publish change to clustering alone. The 1% and 2% loss levels and the topic and rate sweeps use broker-egress loss only.

5.5. Experiment 3: Throughput Under Loss

We measure sustained delivered throughput at QoS 0 using 16 publishers and 8 subscribers at 10 ms one-way delay across loss rates from 0% to 10%, driving each transport to saturation. Every subscriber receives every message, so the broker performs an eight-way fan-out; we report throughput as *unique* messages delivered per second (deliveries divided by eight). The publishers are unpaced, so under loss the delivered rate is whatever the impaired transport can carry, and the offered load differs by transport: under loss the TCP+TLS publishers offer 315–332K msg/s, TCP's 264–273K and QUIC control-only's 228–243K. The publishers' transmitted bytes confirm these rates (271–304 B per message under loss). TCP's publisher hosts run only about 45% busy, so for TCP the broker, at its CPU ceiling, sets the offered rate by how fast it reads its publishers; it takes TCP+TLS publishers faster still, whose hosts run 66–71% busy, a difference we did not isolate. QUIC control-only's publisher hosts run 80–83% busy, so a contribution from the load generator to QUIC's offered rate cannot be excluded. Delay and loss are applied per packet on the forwarding VM (Section 5.2). For TCP, TCP+TLS and the QUIC arms of Fig. 6 the broker process runs near its host's four-core ceiling in every condition (340–390% of 400%), and under loss the subscriber hosts retain headroom (median 91–99% CPU idle), so on impaired links the delivery path rather than the receiver sets the rate. On the clean link the subscribers run 21–42% idle and raw TCP saturates its subscriber host in 11 of 15 runs, which we exclude, so TCP's 0% level rests on 4 runs (the excluded runs delivered slightly more; all 15 average 59.6K, so the retained level slightly understates raw TCP); we read the 0% levels as a joint receiver-and-broker bound. Every cell runs in parallel on three identical groups of broker, publisher, subscriber and forwarding VMs, and ranges quoted “in every group” are over the per-group means. Because the brokers, and for QUIC possibly the generator, bound the offered load, the delivered rates are lower bounds on broker capacity. All Experiment 3 results use the evaluation build except the paced checks at the end of this section, which use the pacing build (Section 4.3).

Fig. 6 shows that all the transports lose throughput as loss rises, differing in magnitude rather than shape. Within QUIC, the client-side publish strategies stay within 1.5% of

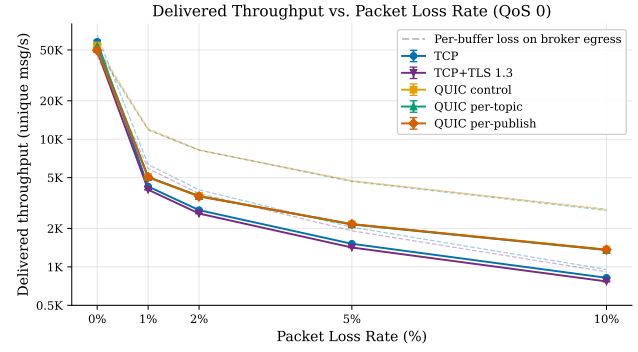


Figure 6: Unique delivered throughput (log scale) vs. packet loss at QoS 0 under per-packet loss (solid), with the eight-way subscriber fan-out normalized out; faded dashed lines show the same experiment under per-buffer loss on the broker's egress (Section 5.2). The QUIC arms (client-side publish strategies; broker delivery per-topic) stay within 1.5% of one another under loss. 95% confidence intervals over 15 runs (14 for per-topic at 5%, 4 for TCP at 0%) are drawn at every point and are frequently narrower than the markers.

one another at every impaired loss rate and within 10% on the clean link. Changing only the broker's delivery from per-topic to control-only, with a control-only client, changes delivered throughput by 6% on the clean link and by at most 1.5% under loss.

Raw TCP's unique delivered throughput falls from 58.0K to 0.82K msg/s between 0% and 10% loss (≈ 71 -fold), the encryption-matched TCP+TLS 1.3 baseline from 49.1K to 0.77K (≈ 64 -fold), and QUIC control-only from 54.3K to 1.37K (≈ 40 -fold). That clean-to-lossy ratio mixes a clean-link level bounded jointly by broker and receiver CPU with the response to loss, so we compare the transports at equal loss instead: QUIC delivers 1.26 $\times$ TCP+TLS's rate at 1% loss, 1.37 $\times$ at 2%, 1.52 $\times$ at 5% and 1.78 $\times$ at 10% (1.19–1.66 $\times$ against raw TCP). Over the loss-bound range from 1% to 10% its throughput falls 1.41 $\times$ less steeply than TCP+TLS's (95% CI 1.40–1.43). Both transports run CUBIC and we did not instrument their loss recovery, so we do not attribute the difference to a specific mechanism.

The comparison is at equal loss but not at equal offered load, since the unpaced publishers offer different loads and every broker runs at its CPU ceiling. To test whether that shapes the lossy rates, we also ran the 1% and 10% loss conditions with the 16 publishers paced to a fixed aggregate rate of 2.5K or 20K msg/s and compared them with unpaced runs. Pacing needs the pacing build (Section 4.3), so both the paced and the unpaced runs of this check use it, and neither is compared with Fig. 6, which uses the evaluation build. Each transport, rate and loss level has 5–6 runs; two runs whose monitoring failed (one lost its broker CPU samples, one its router loss counters) are excluded. Paced at 20K msg/s the brokers use 27–73% CPU out of 400%, yet every transport delivers within 3% of its unpaced rate in the same build (TCP 1.00 and 0.99 of unpaced at 1% and 10% loss, TCP+TLS 0.99 and 0.99, QUIC control-only 1.02 and 1.03). At 2.5K msg/s

the 1% cells deliver everything offered, and at 10% loss even that load exceeds what the loss allows and each transport delivers within 2.5% of its unpaced rate. With the pacing build, at 1% and 10% loss, offered load above the loss-bound rate therefore does not change delivery.

With a matched offered load of 20K msg/s, QUIC delivers 1.28× the rate of TCP+TLS at 1% loss (1.25× unpaced with the pacing build, 1.26× with the evaluation build). At 10% loss the ratio depends on the build: 2.05× paced and 1.97× unpaced in the offered-load check, against 1.78× with the evaluation build. A separate side-by-side comparison of both builds at 10% loss (6 runs per build and transport) gives 1.81× with the evaluation build against 2.02× with the pacing build: the pacing build's QUIC delivers 9% more (1.51K against 1.39K msg/s, 8–9% in every group), its TCP+TLS 1–3% less and its TCP the same. The pacing build's unpaced publishers also offer less load (148K against 245K msg/s for QUIC), and only the pacing build can be paced, so we cannot say whether the gain lies in QUIC's send path or in lighter overload; we report the evaluation build's ratios and treat their magnitude at 10% loss as build-dependent; the builds were not compared at 2% or 5%.

Under loss the delivery path itself narrows, and shedding is where the excess goes: the broker drops what it cannot deliver once a subscriber's queue is full (Table 2), at 10% loss delivering 0.23% of the offered messages over TCP+TLS, 0.30% over TCP and 0.56% over QUIC.

Per-buffer loss on the broker's egress, the method of Experiments 2 and 4, gives a different picture. With it, QUIC delivers 2.06–3.06× TCP+TLS's rate at equal loss and appears to degrade 2.6× more gently from 0% to 10% (Table 3, from the 0% and 10% rows). With per-packet loss on the forwarding VM, QUIC delivers 42–49% of its per-buffer rate and TCP and TCP+TLS 67–86%. Clustered loss therefore flattered QUIC far more than TCP. The loss-bound comparison survives at a smaller magnitude (1.41 against 1.48), and it does not depend on where per-packet loss is applied: keeping the delay on the broker and applying the loss on the forwarding VM gives 1.36 (1.34–1.38; 15 runs per cell), and applying the loss on the broker behind a token-bucket filter that splits every buffer into packets gives 1.39 (1.36–1.41; 6 runs per cell). A model in which clustering acts only by lowering the effective loss rate, with throughput scaling as $1/\sqrt{p}$ [15] and the measured per-buffer drop fractions taken as the effective rates, predicted that correcting it would raise the ratio by a factor of 1.04. It instead fell by a factor of 0.91 (0.89–0.94) with the delay kept on the broker (to 1.36), by 0.95 (0.93–0.98) with the forwarding-VM arm and by 0.93 (0.91–0.96) with the token-bucket arm. The clustering therefore acted through more than the loss rate, and we do not identify the mechanism. A single-connection ablation (one publisher and one subscriber at 10 ms one-way delay, with loss per buffer on the broker's egress; data in the deposit [3]) pointed the same way (disabling QUIC's datagram batching roughly halved its delivered rate under loss), and on that unimpaired single connection TCP+TLS delivered 111.9K msg/s against QUIC's 58.1K.

Table 3

Unique delivered throughput (K msg/s, 15 runs) under per-buffer loss on the broker's egress and under per-packet loss on the forwarding VM, and QUIC control-only's ratio to TCP+TLS at equal loss. G' is the ratio of the two transports' 1%-to-10% degradation, with 95% intervals.

Loss	TCP+TLS		QUIC control-only		QUIC/TLS	
	buffer	packet	buffer	packet	buffer	packet
0%	45.2	49.1	53.7	54.3	1.19	1.11
1%	5.89	4.02	12.16	5.05	2.06	1.26
2%	3.76	2.62	8.26	3.59	2.20	1.37
5%	1.92	1.42	4.75	2.16	2.48	1.52
10%	0.91	0.77	2.80	1.37	3.06	1.78
G' (1%→10%), 95% interval					1.48	1.41
					[1.45, 1.51]	[1.40, 1.43]

Where the delay is applied also matters on the clean link. With the delay on the broker, the broker-egress netem queue of Experiments 2 and 4 (1000 packets) tail-dropped about 0.15% of packets even at 0% loss, and TCP packed several messages into each packet (about 1.1 KB per packet). Enlarging that queue so that it never drops, with nothing else changed, gives one segment per message (about 400 B) and TCP+TLS falls to 25.0K msg/s; with the delay on the forwarding VM instead, TCP packs messages normally (about 1.2 KB per packet) and TCP+TLS delivers 49.1K. The broker's TCP stack disables Nagle's algorithm and writes each message separately; we did not isolate why a large delay queue on the sending host removes the packing. We report the forwarding-VM arm, which avoids both effects; under loss the two delay placements agree within 3% for every configuration except broker per-publish delivery, which with the delay on the broker delivers a further 34% less at 1–2% loss and 18% less at 5%, and the same at 10%.

Encryption carries a predictable throughput cost on the clean link: TCP+TLS runs 15–18% below raw TCP at 0% loss (49.1K against 58.0K unique msg/s over the 4 unsaturated TCP runs, or 59.6K over all 15), a CPU-bound overhead that shrinks to about 6% under loss (4.02K versus 4.26K at 1%, 0.77K versus 0.82K at 10%). QUIC, encrypted throughout, sits above TCP+TLS at 0% loss (54.3K) and 6–9% below raw TCP, so encryption does not explain the loss-response gap.

Broker-side per-publish delivery is the exception to the strategies' near-equality. With a per-publish client and the broker also delivering one stream per message, delivered throughput stays nearly flat at 1.3–2.0K unique msg/s (10–16K deliveries/s) across loss levels. That is far below per-topic delivery on the clean and lightly lossy link (1.74K against 49.6K at 0%, 1.93K against 5.03K at 1%), and the gap closes as loss becomes the binding limit (86% of per-topic at 5%, 94% at 10%). Under loss both arms receive about 135K msg/s from unpaced per-publish publishers, each running at its own stream-credit ceiling (Experiment 4), but per-publish delivery stays near 1.3–2.0K msg/s at every loss level, so

at 1–2% loss it delivers only 38–57% of per-topic’s rate; on the clean link its slow delivery coincides with the publishers running at that ceiling, while with per-topic delivery they offer about 50K. Pacing the publishers separates delivery cost from overload. For this we used the pacing build. Its unpaced per-topic and control-only deliveries on the clean link run 10–12% below the evaluation build’s, so we compare delivery strategies only among pacing-build runs. At 0% loss, per-publish delivery keeps up with the eight-way fan-out up to 40K deliveries/s, then stalls near 72K deliveries/s (about 9K msg/s per subscriber), consistent with a per-subscriber stream-credit ceiling at this round trip; we did not vary the stream limit or the round trip for delivery. Below that ceiling it costs $1.17\times$ the broker CPU per delivered message of per-topic delivery at 10K deliveries/s, $1.33\times$ at 20K and $1.41\times$ at 40K. It sends about one datagram per message (1.06–1.08 broker packets per delivered message over 10–40K deliveries/s, against 0.89 falling to 0.54 for per-topic delivery over the same range). Under 1% loss its packets per delivered message fall to 0.35–0.93 as messages share datagrams, yet it still costs 1.15 – $1.33\times$ per-topic’s CPU per delivered message, so the per-datagram cost does not fully explain the premium. When paced at 1% loss, every strategy saturates near 5K msg/s per subscriber (per-publish 4.6K, per-topic and control-only 5.1K), so there loss rather than credit sets most of the ceiling. Unpaced, per-topic delivery reaches 93% of its paced ceiling, while per-publish delivery reaches only 17% of its own (about 0.8K msg/s per subscriber, the same as on the clean link in the pacing build, where unpaced per-publish delivery runs about 55% below the evaluation build’s 1.74K), so its unpaced collapse comes from overload, not loss. We did not test whether batching per-publish frames removes the premium.

5.6. Experiment 4: What Limits a Single Connection

Experiment 3 measures the broker at saturation, where many client connections share it and, under loss, the delivery path sets the rate. A single MQTT client cannot spread its own load that way: the protocol ties each connection to a client identifier, and a second connection presenting an identifier already in use causes the broker to disconnect the first (Section 6.3). Short of provisioning further identities, everything one client publishes therefore travels over a single QUIC connection, and that connection’s limits bound what the client can send. Two of those limits are transport parameters an implementation chooses, the number of streams the peer lets it open at once and the flow-control window of each stream; above them we also find a rate ceiling that we did not instrument. The three strategies differ sharply in what one connection can push, and this experiment asks why. We vary the number of topics (1, 2, 4, 8, 16) on a *single* connection, and then vary the two QUIC transport limits the strategies interact with: the number of streams a peer may keep open at once, and the per-stream receive window. The broker’s delivery strategy matches the client’s in each arm. Unless stated, cells use Experiment 2’s delay and broker-egress loss method, with

25 ms one-way delay and 2% loss, one of Experiment 2’s loss levels; publisher and subscriber run on separate VMs and the publishers are unpaced. Loss is not this experiment’s variable. Because *tc-netem* shapes the broker’s egress, the publisher’s data travels an unimpaired path, and the delay and loss apply to the acknowledgements and flow-control credit returning to it (and to the subscriber’s deliveries); since the delay applies in one direction only, the publisher’s round trip equals the 25 ms one-way delay. Cells run at 0% loss check that the loss on that returning path does not shape the limits: control-only at the default 256 KB window, where the window binds, sends 32.97K msg/s at 0% against 32.90–33.23K in the equivalent 2% cells, and removing the loss does not raise the per-connection ceiling (below). No per-publish cell was run at 0% loss, so the effect of loss on its credit-bound rate is untested; QUIC’s credit frames carry cumulative limits [9], so a lost update delays credit but does not forfeit it.

Varying topic count alone shows the gap between the strategies on a single connection. Control-only sustains about 33K msg/s at every topic count and per-publish about 3.8K, roughly ninefold below it. Per-topic is the only strategy that moves with topic count, and it moves once: it matches control-only on one topic (33.1 against 32.7K), where it opens the same single data stream, reaches 56.0K at two topics, and then stays flat to sixteen (53.9K). Adding streams beyond the second buys nothing, which already rules out an explanation in terms of write concurrency scaling with stream count.

Fig. 7 identifies the limits. Per-publish is bounded by stream credit. Sweeping the limit over 25, 100, 250 and 1000 streams moves its rate to 0.93, 3.75, 9.19 and 30.87K msg/s, and the streams it keeps in flight, the rate multiplied by the round trip, stay at 93%, 94%, 93% and 78% of the limit. Holding the limit at 100 and sweeping the delay over 10, 25 and 50 ms leaves that count at 93.2, 94.4 and 95.4 streams: the rate falls as $1/\text{RTT}$, which is what a credit ceiling does and what a per-message processing cost would not. Control-only and per-topic, needing one and eight streams, barely respond to the stream limit itself: raising it from 100 to 1000 moves control-only by 0.8% (33.13 to 32.85K) and per-topic by 7.3% (50.96 to 54.67K, $p = 0.09$).

Control-only is bounded by the per-stream receive window. Halving it to 128 KB halves the rate to 16.79K and raising it to 1 MB lifts it to 54.30K. Per-topic on one topic, which also opens a single data stream, tracks it to within 0.2% wherever the window binds (16.76 against 16.79K at 128 KB, 32.92 against 32.90K at 256 KB). At 1 MB the window no longer binds either arm and they separate, per-topic on one topic reaching 49.12K against control-only’s 54.30K ($p = 1 \times 10^{-6}$); the two are single-stream but not identical, since control-only carries its data on the bidirectional control stream while per-topic opens a unidirectional one, and that difference surfaces only once both are pressed against the ceiling below. Across the six cells of the window and round-trip sweeps in which the window binds, spanning a factor of two in window and a factor of two in round trip, and including one configuration measured on two VM groups, the implied per-message cost on the stream is constant at 312.8 ± 3.0 B (a

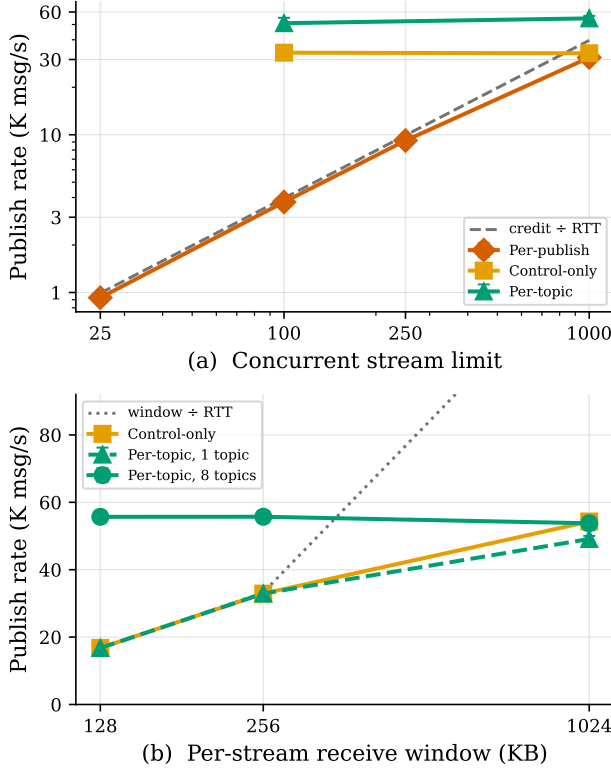


Figure 7: The two transport limits behind those rates, 8 topics unless noted. (a) Per-publish opens one stream per message, so its rate follows the concurrent stream limit divided by the round trip (dashed) across a fortyfold range, while control-only and per-topic, which open one and eight streams, do not respond to it. (b) Control-only and per-topic on a single topic both open one data stream and coincide while the per-stream receive window binds, following window divided by round trip (dotted, using the 313 B per-message stream cost measured from these runs); at 1 MB the window no longer binds and they separate, only control-only reaching the rate ceiling near 54K msg/s. Per-topic on eight topics is already at that ceiling and is insensitive to the window. 95% confidence intervals over 10 runs per point.

1.0% spread), consistent with a 256 B payload plus MQTT and QUIC stream framing. That constancy is the evidence; the absolute value is derived from the same measurements and is not an independent prediction. Per-topic on eight topics has eight such windows and is therefore insensitive to the setting (55.7, 55.7 and 53.8K).

Above those two limits sits a third, a rate ceiling on one connection near 54K msg/s: the cells it bounds deliver 51.0–56.0K at 25 ms. Removing the loss does not raise it (control-only at a 1 MB window sends 52.1K at 0% against 54.3K at 2%, and per-topic’s 53.6K at 0% lies within the 51.0–55.7K of its equivalent 2% cells). It is also roughly independent of the round trip, reaching 57.8K at 10 ms, so it is neither a window nor a credit effect. We did not instrument the client further, and attribute it only to per-connection processing.

The strategies therefore do not differ in what they can carry; they differ in which limit they meet first. Per-publish meets the stream limit, control-only the per-stream window,

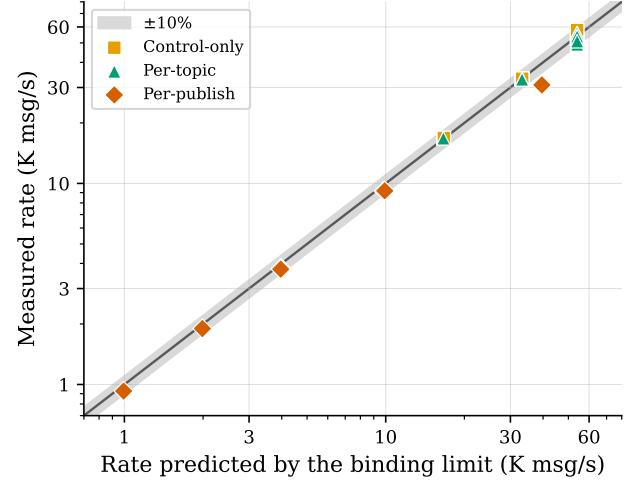


Figure 8: Every publish-rate cell of Experiment 4 against the rate its binding limit predicts: stream credit divided by the round trip for per-publish, and the stream count times the per-stream window divided by the round trip for the others, each capped at the 54K msg/s per-connection ceiling. The 39 cells span a sixtyfold range of measured rate and sit on the diagonal with a median absolute error of 2.3%. The two constants in the model, the 313 B per-message stream cost and the ceiling, are measured from these runs rather than fitted per cell.

and per-topic, which escapes the window with its second stream, meets the per-connection ceiling. Lift each in turn and they move toward that ceiling: control-only reaches 54.3K at a 1 MB window and per-topic already sits at 53.8K, while per-publish, at 30.9K with a limit of 1000, is still climbing and would need a larger limit still to reach it. The ninefold gap between per-publish and control-only at our defaults, and per-topic’s $\approx 1.6\times$ advantage over control-only, are both consequences of two constants we chose, not properties of the mappings. Fig. 8 states the position compactly: across all 39 publish-rate cells, spanning stream limits from 25 to 1000, windows from 128 KB to 1 MB, delays from 10 to 50 ms and both loss settings, the measured rate is within a median 2.3% of whichever limit binds, and the strategy identity adds little once that limit is known. The largest departures are per-publish at a 1000-stream limit, 22% below the credit line as it approaches the ceiling; per-topic on one topic at a 1 MB window, 9% below the ceiling, the single-stream separation discussed above; per-publish at a 250-stream limit, 7% below its credit line, as per-publish sits 5–7% below it at every limit up to 250; and control-only at 10 ms, 7% above a ceiling that is itself slightly round-trip dependent.

Two caveats bound this. The limits are ours: `mqtt5` advertises a 100-stream limit, which is the default of the QUIC library it builds on [19], and a 256 KB per-stream window, and an implementation choosing differently would order the strategies differently. And the finding is about one connection. At broker capacity the shared delivery path sets the rate, which is why Experiment 3’s client-side strategies differ by at most 1.5% under loss and 10% on the clean link; we report the publish rate here because a single subscriber

connection receives only about 3.4K msg/s under control-only and per-topic and 2.1K under per-publish, its host staying 97% CPU idle, so the delivered rate in this one-to-one setup measures the impaired delivery path rather than the send-side strategy.

6. Discussion

6.1. Strategy Selection Guidelines

Our results suggest the following guidelines for practitioners selecting a stream mapping strategy:

Use control-only as the default. Adding no per-stream packet overhead, it has the lowest median and 90th-percentile tail latency at 5% loss, 8 topics and 500 msg/s (132 and 162 ms; 108 and 138 ms under per-packet loss). At 1% loss and 500 msg/s both multistream arms beat it, and at 125 msg/s under 5% loss per-publish does (96 versus 116 ms); at 1% loss and 1000 msg/s control-only is again lowest (61 against 70 and 82 ms), so which strategy has the lowest tail depends on both loss and rate. As the client publish strategy it matches the other QUIC arms' delivered throughput within 1.5% under loss and 10% on the clean link; as the broker's delivery strategy with a control-only client it is within 1.5% under loss and 6% lower on the clean link (Experiment 3). It inherits TLS 1.3 in a one-round-trip handshake that matches raw TCP and saves a round trip over TCP+TLS at delays of 25 ms and above (Experiment 1). Its one throughput concession is on a single connection, where per-topic sends about 1.6–1.7× faster once a second topic is in play; Experiment 4 shows that gap is the per-stream receive window, and raising it removes the gap. Like the other strategies, it has occasional outlier runs (Experiment 2).

Consider per-topic when one client must publish at high rate across several topics and the per-stream window cannot be raised. With the default 256 KB per-stream window, per-topic reaches the per-connection ceiling once a second topic is in play (about 56K against control-only's 33K msg/s; Experiment 4), because each topic gets its own window; raising control-only's window closes that gap. Under broker-egress loss at 500 msg/s its tail sits between the other two, below control-only at 1% loss (55.8 against 58.3 ms) and above it at 5% (164 against 132 ms); under per-packet 5% loss it ties per-publish (141 against 143 ms), both above control-only (108 ms). In the 1%-loss topic sweep it isolates spike timing only at two topics and reaches single-stream coupling from four topics on (Fig. 4).

Use per-publish for low-rate or light-loss channels where per-message isolation matters. It isolates spikes best at every rate we tested under 1% loss. It has the lowest tail at 1% loss up to 500 msg/s (54.0 ms at the reference point), and at 125 msg/s under 5% loss (96 ms). On a single connection its publish rate tracks the concurrent stream limit divided by the round trip, roughly ninefold below control-only at our 100-stream limit and 256 KB window. That rate rises nearly in proportion as the limit is raised, but only toward the per-connection ceiling of about 54K msg/s (30.9K at a 1000-stream limit; Experiment 4). As the broker's delivery strategy

it is capped per subscriber near 9K msg/s on the clean link at a 10 ms round trip (under 1% loss all strategies saturate near 5K), costs 1.2–1.4× the broker CPU per delivered message of per-topic delivery at 10–40K deliveries/s, and under unpaced overload delivers a small fraction of what per-topic delivery achieves on a clean link (Experiment 3). Size the stream limit to the per-subscriber message rate.

6.2. Loss Resilience and the Cost of Encryption

Because our QUIC arms are encrypted, we compare them against an encryption-matched TCP+TLS 1.3 baseline, so the difference reflects the transport rather than the cipher. Under per-packet loss QUIC delivers 1.26× TCP+TLS's rate at 1% loss, growing to 1.78× at 10%, and over that range its throughput falls 1.41× less steeply. Both transports run CUBIC and we did not instrument their loss recovery, so the mechanism remains open. With the pacing build, pacing all three transports to a matched offered load far below the broker's CPU ceiling left their rates at 1% and 10% loss within 3% of unpaced, so in the pacing build the gap is not an artifact of unequal load. The evaluation build, whose ratios we report, has no rate pacing and could not be checked this way, and at 10% loss the gap depends on the build (2.02× against 1.81× when both are run side by side). The advantage is real but smaller than the 2.06–3.06× that per-buffer loss emulation produced with the evaluation build. Under per-buffer loss QUIC also appears to degrade 2.6× more gently from 0% to 10% loss, but that ratio rests on the method and on a clean-link level bounded jointly by broker and receiver CPU; with per-packet loss the same 0-to-10% ratio is 1.61 with the delay on the forwarding VM, but 0.82 with the delay on the broker and the loss on the forwarding VM and 0.86 behind the broker's token-bucket filter, which is why we compare at equal loss. On the clean link the ordering depends on configuration: at fan-out saturation QUIC delivers 54.3K msg/s against TCP+TLS's 49.1K, whereas on a single unimpaired connection at 10 ms TCP+TLS reaches about 112K while QUIC stays near 58K, at the per-connection ceiling of Experiment 4. The benefit is therefore conditional on the loss a deployment sees and on how its traffic is spread across connections. Encryption is a predictable cost: TCP+TLS runs 15–18% below raw TCP on the clean link and about 6% below under loss.

6.3. The Cost of Parallel Connections

A TCP client can obtain per-topic concurrency simply by opening several connections, and doing so carries a bandwidth consequence: each connection is a separate congestion controller, so N of them converge to roughly N times a single flow's share of a congested link [11, 4]. For a generic TCP application this is a real alternative to stream multiplexing.

MQTT forbids it. The protocol ties each connection to a client identifier, and a second connection presenting an identifier already in use causes the broker to disconnect the first (normative statement [MQTT-3.1.4-3] [18]). An MQTT device therefore cannot open the N connections that would claim the larger share; it would have to provision N distinct identities, which is a fleet-management and, on managed

platforms, a billing cost rather than a socket call. Managed IoT platforms enforce this rule in practice; Azure IoT Hub, for one, permits a single active connection per device and drops the existing one when a second presents the same identity [16]. Stream multiplexing is therefore the concurrency mechanism actually available to an MQTT client: it supplies per-topic streams inside the single identity the protocol expects.

6.4. HOL Blocking: Transport and Stream Mapping

At 5% loss, and at 2% for control-only and per-topic, our HOL blocking results locate the benefit in the transport rather than in stream mapping. TCP's single ordered byte stream couples every topic (351 ms median p99 at 5%; 458 ms in the per-packet variant, whose broker-side delay queue also changes TCP's packetization; Section 5.5). QUIC control-only couples topics just as strongly at 1% loss (Fig. 4), yet it and every other QUIC strategy keep the median tail below 200 ms at the 8-topic, 500 msg/s reference point; we did not isolate which transport difference is responsible. At 1% loss control-only is indistinguishable from TCP (58.3 against 61.5 ms, $p = 0.12$), while finer stream mapping lowers the tail significantly (per-topic 55.8, per-publish 54.0 ms; $p = 0.007$ and $p = 0.003$ against control-only). Stream granularity is a tradeoff knob within QUIC. Finer streams shrink the set of messages one loss can block, which decorrelates spike timing: per-publish isolates most, while in the 1%-loss sweeps per-topic isolates only at two topics and at 2000 msg/s. But finer streams also add framing packets. Under per-buffer 5% loss, per-publish sent about twice control-only's packets on the delivery path, had 2.5 times its congestion events and a smaller congestion window, and had the highest tail (195 against 132 ms). Per-topic added only about 9% more packets, yet its median sat midway between the two (164 ms). Under per-packet loss per-publish no longer trails per-topic (143 against 141 ms) although it still sends about 1.5 times per-topic's packets, and both sit about 30% above control-only. Packet count therefore does not fully explain the ordering under either method; QUIC loss detection is per-connection rather than per-stream [8], and we could not separate per-stream recovery effects from packet count.

6.5. Threats to Validity

Our results come from one implementation, evaluated by its author, on one QUIC library (Quinn 0.11) whose default 100-stream limit, together with our 256 KB per-stream window, decides the single-connection ordering (Experiment 4). Loss is random and independent from packet to packet (Bernoulli), without bursts, and applies only to broker-to-client traffic, so the publish path is never lossy and bursty loss is untested. Experiment 2 applies loss per kernel buffer on the broker's egress, a method Experiment 3 shows inflates QUIC's throughput advantage; Experiment 4's publish rates are measured on an unimpaired publish path, with the loss on the returning acknowledgements and credit; its 0%-loss cells show little effect on the window-bound and ceiling cells, and no per-publish cell was run at 0%. The two 5%-loss per-publish delivery cells that Experiment 2 cites carry

per-buffer loss on their delivery path. Only Experiment 2's reference cell at 0% and 5% was also run with per-packet loss; there QUIC's tail advantage over TCP did not shrink but the per-publish tail penalty over per-topic disappeared, so its other cells may carry similar distortions. All VMs share one GCP zone with emulated delay and CUBIC, and all runs use 256-byte QoS 0 payloads, so the acknowledgement traffic of QoS 1 and 2 is untested. Brokers are CPU-bound at saturation, QUIC control-only's publisher hosts run 80–83% busy under loss, and TCP's clean-link level in Experiment 3 rests on the 4 runs whose subscriber did not saturate. At 10% loss QUIC's advantage over TCP+TLS depends on the implementation build (1.81× with the evaluation build against 2.02× with the pacing build, side by side), and we did not identify the change responsible. The pacing build produces the offered-load check (1% and 10% loss only), the broker per-publish CPU premium, per-subscriber ceiling and paced ceilings, their unpaced comparisons and the side-by-side 10% comparison; its unpaced clean-link per-topic and control-only deliveries run 10–12% below the evaluation build's, and its unpaced per-publish delivery 55–59% below (0% and 1% loss). Experiment 2 co-locates publisher and subscriber, and the broker's delivery mapping is varied independently of the client's only in Experiment 3's fan-out configuration. Fifteen runs per arm cannot estimate the rate of rare tail outliers.

6.6. Future Work

This work opens several directions. Per-publish delivery could pack the frames of several streams into shared datagrams, which would show whether its broker CPU premium is inherent to one stream per message or a consequence of sending one datagram per message. Evaluating the strategies at QoS 1 and 2, where acknowledgements share the impaired path, and under bursty loss would extend the comparison to conditions common in IoT deployments, and long-running measurements could estimate how often tail outliers occur and whether they depend on the stream mapping. Because the tail ordering of the strategies reverses with both loss and rate, a client or broker could select stream granularity adaptively from the loss and rate it observes. Finally, constrained devices, 0-RTT resumption and connection migration would test QUIC's other advertised benefits for IoT.

7. Conclusion

We implemented and evaluated three stream mapping strategies for running MQTT over QUIC, including broker-autonomous per-topic delivery-stream mapping absent from existing brokers. At 5% loss the transport, not stream mapping, sets the level of HOL blocking: control-only and per-topic keep the median per-topic tail below TCP at 2% and 5% loss, and per-publish does so at 5% (at 2% its 82.6 ms is not distinguishable from TCP's 89.0 ms, $p = 0.06$), and at 5% control-only holds 132 ms against TCP's 351 ms, and 108 against 458 ms in the per-packet variant. Within QUIC, control-only adds the least packet overhead and has the lowest typical and 90th-percentile tail at 5% loss and 500 msg/s; at

1% loss and 500 msg/s both multistream arms come out ahead, and at 125 msg/s under 5% loss per-publish does.

Under loss QUIC delivers more than an encryption-matched TCP+TLS 1.3 baseline, 1.26 $\times$ at 1% loss rising to 1.78 $\times$ at 10%, once the emulated loss removes single packets; the evaluation build that produced these ratios could not be paced, and at 10% loss the ratio depends on the implementation build, rising to 1.97–2.05 $\times$ with the pacing build (Sections 5.5 and 6.5). Loss applied per kernel buffer on the sending host inflated that advantage to 2–3 $\times$; we show how to detect this from the qdisc’s own counters and how to avoid it with a forwarding hop. Broker-side per-publish delivery has a per-subscriber ceiling on the clean link consistent with stream credit, costs more broker CPU per delivered message than per-topic delivery, and under unpaced load at light loss delivers a fraction of per-topic delivery’s rate (38% at 1% loss), a gap that closes as loss becomes the binding limit. On a single connection the strategies differ ninefold, but that difference is set by two QUIC transport constants, the concurrent stream limit and the per-stream receive window, rather than by the mappings.

For practitioners: use control-only as the default for the lowest typical tail latency at heavy loss and moderate-to-high rate (5% loss, 500 msg/s); consider per-topic when a single client must publish beyond control-only’s window-bound rate (about 33K msg/s here) across several topics and the per-stream window cannot be raised; use per-publish for low-rate or light-loss channels that need per-message isolation, sizing the stream limit to the per-subscriber message rate. Whether QUIC’s throughput gain matters depends on the loss a deployment sees: on a clean link both transports are bound jointly by broker and receiver CPU, and the gap between them depends on configuration. The implementation is available as open-source software.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Data Availability

All experimental data reported in this study, together with the scripts that produced and analyzed them, are archived at Zenodo [3]. The implementation, analysis scripts, figure generators and experiment infrastructure are also available in the project repository at <https://github.com/LabOverWire/mqtt-lib>.

CRedit authorship contribution statement

Fabrizio Bracht: Conceptualization, Methodology, Software, Investigation, Data curation, Writing – original draft, Writing – review & editing, Visualization.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the author used Claude (Anthropic) in order to assist with experiment design and scripting, data analysis, and manuscript drafting and editing. After using this tool, the author reviewed and edited the content as needed and takes full responsibility for the content of the published article.

References

- [1] Belshe, M., Peon, R., Thomson, M., 2015. Hypertext transfer protocol version 2 (HTTP/2). RFC 7540.
- [2] Bishop, M., 2022. HTTP/3. RFC 9114.
- [3] Bracht, F., 2026. Experiment data for “Evaluating Stream Mapping Strategies for MQTT over QUIC”. doi:10.5281/zenodo.19098819.
- [4] Chiu, D.M., Jain, R., 1989. Analysis of the increase and decrease algorithms for congestion avoidance in computer networks. *Computer Networks and ISDN Systems* 17, 1–14. doi:10.1016/0169-7552(89)90019-6.
- [5] EMQ Technologies, 2024a. EMQX: An open-source, highly scalable MQTT broker. URL: <https://www.emqx.io/>.
- [6] EMQ Technologies, 2024b. NanoMQ: MQTT over QUIC bridge. URL: <https://nanomq.io/docs/en/latest/bridges/quic-bridge.html>.
- [7] Fernández, F., Zverev, M., Garrido, P., et al., 2020. And QUIC meets IoT: Performance assessment of MQTT over QUIC, in: *IEEE International Conference on Wireless and Mobile Computing, Networking and Communications (WiMob)*.
- [8] Iyengar, J., Swett, I., 2021. QUIC loss detection and congestion control. RFC 9002.
- [9] Iyengar, J., Thomson, M., 2021. QUIC: A UDP-based multiplexed and secure transport. RFC 9000.
- [10] Kakhki, A.M., Jero, S., Choffnes, D., Nita-Rotaru, C., Mislove, A., 2017. Taking a long look at QUIC: An approach for rigorous evaluation of rapidly evolving transport protocols, in: *Proceedings of the 2017 Internet Measurement Conference (IMC)*, ACM.
- [11] Kühlewind, M., Trammell, B., 2022. Applicability of the QUIC transport protocol. RFC 9308.
- [12] Kumar, P., Dezfooli, B., 2019. Implementation and analysis of QUIC for MQTT. *Computer Networks* 150, 28–45. doi:10.1016/j.comnet.2018.12.012.
- [13] Langley, A., Riddoch, A., et al., 2017. The QUIC transport protocol: Design and internet-scale deployment. *Proceedings of ACM SIGCOMM*.
- [14] Marx, R., et al., 2020. Same standards, different decisions: A study of QUIC and HTTP/3 implementation diversity, in: *Workshop on the Evolution, Performance, and Interoperability of QUIC (EPIQ)*.
- [15] Mathis, M., Semke, J., Mahdavi, J., Ott, T., 1997. The macroscopic behavior of the TCP congestion avoidance algorithm. *ACM SIGCOMM Computer Communication Review* 27, 67–82.
- [16] Microsoft, 2024. Communicate with your IoT hub using the MQTT protocol. Microsoft Azure IoT Hub documentation. URL: <https://learn.microsoft.com/en-us/azure/iot-hub/iot-hub-mqtt-support>.
- [17] Mishra, B., Kertesz, A., 2020. The use of MQTT in M2M and IoT systems: A survey. *IEEE Access* 8, 201071–201086. doi:10.1109/ACCESS.2020.3035849.
- [18] OASIS, 2019. MQTT version 5.0. OASIS Standard. URL: <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>.
- [19] Ochtman, D., Saunders, B., et al., 2024. Quinn: QUIC transport protocol implementation for Rust. URL: <https://github.com/quinn-rs/quinn>.
- [20] Ott, J., Even, R., Perkins, C., Singh, V., 2017. RTP over QUIC. Internet-Draft draft-rtpfolks-quic-rtp-over-quic-01, IETF. URL: <https://datatracker.ietf.org/doc/html/draft-rtpfolks-quic-rtp-over-quic>.

- [21] Scharf, M., Kiesel, S., 2006. Head-of-line blocking in TCP and SCTP: Analysis and measurements, in: Proceedings of the IEEE Global Telecommunications Conference (GLOBECOM 2006), IEEE, San Francisco, CA, USA. doi:10.1109/GLOCOM.2006.333.
- [22] Thomson, M., Turner, S., 2021. Using TLS to secure QUIC. RFC 9001.
- [23] Yang, W., 2024. Spec MQTT-next: A mapping of MQTT to the QUIC transport protocol. OASIS MQTT Technical Committee, document 71729. URL: https://groups.oasis-open.org/higherlogic/ws/public/download/71729/oasis_mqtt_over_quic.pdf.
- [24] Yu, A., Benson, T.A., 2021. Dissecting performance of production QUIC, in: Proceedings of the Web Conference 2021 (WWW), ACM.